MINISTERE DE L'ENSEIGNEMENT
SUPERIEUR ET DE LA RECHERCHE
(MESR)

REPUBLIQUE TOGOLAISE
Travail-Liberté-Patrie

ECOLE SUPERIEURE DE GESTION D'INFORMATIQUE ET DES SCIENCES

RAPPORT DE PROJET

CONCEPTION D'UN RESEAU SOCIAL EN DJANGO : CONNECTX

Programme	Master 1 IA & Big Data
Cours	PYTHON WEB : DJANGO
Chargé de cours	M. NYAKU
Auteurs	GBOSSOU Folly Sitou Carlo POTCHO ESSOSSOLAM Maxime
Année académique	2025 – 2026
Date de remise	

Résumé

ConnectX est une plateforme de réseau social avancé développée dans le cadre du module Python/Django du Master 1 en Intelligence Artificielle et Big Data. Le projet vise à concevoir et implémenter une application web complète reproduisant les fonctionnalités essentielles des grands réseaux sociaux modernes, tout en mettant en pratique les compétences acquises en développement web Python.

La plateforme permet aux utilisateurs de créer un compte sécurisé avec vérification d'email obligatoire, de publier du contenu textuel et multimédia jusqu'à 5 images par publication, de suivre d'autres utilisateurs dans un modèle d'abonnement asymétrique, d'échanger des messages privés en temps réel via WebSocket, de publier des stories éphémères de 24 heures, et de recevoir des notifications instantanées (toasts + badges) pour chaque interaction sociale.

L'application repose sur une architecture ASGI pilotée par Daphne, couplant la couche HTTP classique de Django et la couche WebSocket de Django Channels via un même point d'entrée. La base de données PostgreSQL stocke toutes les entités métier qui héritent d'un modèle abstrait commun (BaseModel) implémentant l'audit complet et le soft delete. En production, Redis est utilisé comme channel layer pour permettre la communication WebSocket entre processus. Le tout est containerisé avec Docker et déployé sur Hetzner Cloud via Coolify, accessible à l'adresse https://connectx.gfolly.com.

Technologies : Python 3.12, Django 5, Django Channels, Daphne (ASGI), PostgreSQL, Redis, Bootstrap 5, Jazzmin, WhiteNoise, Brevo SMTP, Docker, Hetzner Cloud, Coolify.

Table des Matières

1. Introduction
1.1 Contexte et problématique
1.2 Objectifs du projet
1.3 Plan du rapport

2. Cahier des Charges
2.1 Besoins fonctionnels
2.2 Besoins non fonctionnels
2.3 Contraintes techniques
2.4 Public cible

3. Conception et Architecture
3.1 Architecture globale et modules
3.2 Modèle de données détaillé
3.3 Architecture ASGI et WebSocket
3.4 Choix technologiques et justifications

4. Développement
4.1 Structure du projet et découpage modulaire
4.2 Méthodologie de travail
4.3 Module accounts — Authentification et profils
4.4 Module posts — Publications et fil d'actualité
4.5 Module social — Abonnements et découverte
4.6 Module chat — Messagerie en temps réel
4.7 Module notifications — Alertes instantanées
4.8 Module stories — Contenu éphémère
4.9 Interface d'administration Jazzmin
4.10 Problèmes rencontrés et solutions

5. Déploiement et DevOps
5.1 Infrastructure Cloud — Hetzner + Coolify
5.2 Containerisation Docker
5.3 Configuration des services en production
5.4 Problèmes de déploiement résolus

6. Sécurité de l'Application
6.1 Workflow de vérification d'email
6.2 Gestion et protection des mots de passe
6.3 Protections CSRF, XSS et reverse proxy
6.4 Sécurité WebSocket et UUID publics

7. Tests et Validation
7.1 Méthodologie de test
7.2 Résultats des tests fonctionnels
7.3 Améliorations possibles

8. Conclusion et Perspectives

Annexes

1. Introduction

1.1 Contexte et problématique

Les réseaux sociaux occupent aujourd'hui une place centrale dans la communication numérique. Qu'il s'agisse d'Instagram, de Twitter/X, de Facebook ou de LinkedIn, ces plateformes ont en commun une architecture web complexe capable de gérer de multiples interactions sociales en temps réel : publications, réactions, messagerie instantanée, notifications push, stories éphémères. La conception d'un tel système constitue un excellent exercice d'ingénierie logicielle, car elle mobilise simultanément de nombreuses compétences : modélisation de données relationnelles complexes, programmation orientée objet, protocoles de communication réseau, gestion de l'authentification et de la sécurité, optimisation des requêtes, et conception d'interfaces utilisateur réactives.

Dans ce contexte, le présent projet — ConnectX — répond à la problématique suivante : comment concevoir et développer, avec le framework Django, une plateforme de réseau social fonctionnelle qui gère efficacement des relations utilisateurs complexes tout en offrant des interactions en temps réel ? Cette problématique soulève plusieurs questions techniques fondamentales : comment modéliser proprement des relations many-to-many comme les abonnements asymétriques ou les likes en garantissant la cohérence des données ? Comment implémenter la communication bidirectionnelle persistante avec Django, traditionnellement synchrone et basé sur le modèle requête-réponse HTTP ? Comment assurer la sécurité des données et des échanges dans un contexte multi-utilisateurs ?

Le sujet impose l'utilisation de Django Channels, une extension officielle de Django permettant de gérer des protocoles asynchrones comme les WebSockets. Ce choix technologique impose de sortir du modèle requête-réponse classique (WSGI) pour adopter une architecture ASGI (Asynchronous Server Gateway Interface), ce qui constitue un vrai défi de conception pour des étudiants de Master 1 : il faut comprendre la programmation asynchrone Python (async/await), les notions de consumer, channel group et channel layer, et gérer la coexistence de deux paradigmes de communication au sein d'une même application.

1.2 Objectifs du projet

Le projet ConnectX poursuit les objectifs techniques et fonctionnels suivants :

- Implémenter un système complet de publications avec support multimédia : texte (2 000 caractères max) et jusqu'à 5 images avec validation côté client et côté serveur.
- Gérer des relations d'abonnement asymétriques entre utilisateurs (follow/unfollow) avec soft delete pour préserver l'historique.
- Développer un système de messagerie privée fonctionnel avec couche HTTP pour l'historique et couche WebSocket pour le temps réel, incluant les indicateurs de frappe et de présence.
- Mettre en place un système de notifications push déclenché automatiquement par les signaux Django sur quatre types d'événements : like, commentaire, abonnement, message.
- Implémenter les stories éphémères : contenu qui expire automatiquement après 24 heures sans tâche planifiée, par filtrage ORM sur le champ expires_at.
- Assurer la sécurité de l'application : authentification avec vérification d'email obligatoire (is_active=False jusqu'à validation), tokens à usage unique, protections CSRF/XSS, UUID publics dans les URLs.

- Déployer l'application en production sur infrastructure Cloud (Hetzner + Coolify) avec containerisation Docker, serveur ASGI Daphne, et Redis comme channel layer de production.
- Concevoir une architecture Django propre, extensible et maintenable : sept apps indépendantes, modèle BaseModel abstrait commun, pattern soft delete et audit trail.

1.3 Plan du rapport

Ce rapport est structuré en huit parties. La section 2 présente le cahier des charges complet. La section 3 détaille l'architecture et les choix de conception. La section 4 décrit le développement module par module, avec des extraits de code commentés tirés du code source réel. La section 5 couvre le déploiement et les pratiques DevOps. La section 6 approfondit les mécanismes de sécurité. La section 7 présente les tests et leur validation. La section 8 conclut avec le bilan et les perspectives. Des annexes avec le code source complet des composants clés complètent le document.

2. Cahier des Charges

2.1 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que le système doit faire du point de vue de l'utilisateur final. Ils sont organisés en sept modules fonctionnels correspondant aux apps Django du projet.

BF1 — Gestion des comptes utilisateurs

- Inscription avec nom d'utilisateur unique et adresse e-mail unique, avec validation des mots de passe (similarité, longueur minimale, mots de passe communs, entièrement numérique).
- Vérification d'email obligatoire avant toute connexion : le compte est créé avec is_active=False et ne s'active qu'au clic sur le lien envoyé par email.
- Authentification par e-mail et mot de passe (USERNAME_FIELD = 'email') avec messages d'erreur personnalisés distinguant compte non activé et identifiants incorrects.
- Réinitialisation de mot de passe par email avec tokens à usage unique (PasswordResetTokenGenerator Django).
- Changement de mot de passe depuis les paramètres du profil, exigeant la connaissance du mot de passe actuel.
- Personnalisation du profil : biographie, avatar (upload_to='avatars/'), image de couverture (upload_to='covers/'), localisation, site web.
- Consultation du profil public de tout utilisateur, avec compteurs d'abonnés, d'abonnements et de publications.

BF2 — Publications

- Création d'une publication composée d'un texte (2 000 caractères max) et jusqu'à 5 images (JPEG, PNG, GIF, WebP ; 5 Mo max par image).
- Validation à deux niveaux : côté client (JavaScript, prévisualisation) et côté serveur (content_type, taille, nombre MAX_IMAGES = 5).
- Fil d'actualité chronologique (order_by('-created_at')) des publications des utilisateurs suivis et de l'utilisateur connecté.
- Like / unlike avec réponse JSON pour mise à jour AJAX instantanée sans rechargement de page. Soft delete sur Like pour préserver l'historique.
- Commentaires sur chaque publication, avec suppression logicielle (soft delete).
- Suppression de ses propres publications (soft delete : is_deleted=True, deleted_at, deleted_by).

BF3 — Abonnements (Follow)

- Follow / unfollow avec réponse JSON pour mise à jour AJAX. Soft delete sur Follow pour préserver l'historique et permettre le re-follow.
- Contrainte d'unicité en base (unique_together = ('follower', 'following')) pour empêcher les doublons.
- Prévention de l'auto-follow : vérification if target == request.user dans FollowToggleView.
- Suggestions d'utilisateurs : liste de tous ceux que l'utilisateur ne suit pas encore, excluant les comptes staff et superuser.
- Listes détaillées des abonnés et abonnements consultables sur chaque profil public.

BF4 — Messagerie privée

- Démarrage ou retrouvaille d'une conversation privée avec n'importe quel utilisateur (StartConversationView crée ou retrouve la conversation existante).
- Messagerie en temps réel via WebSocket (ChatConsumer) : envoi, réception, persistance en base (database_sync_to_async).
- Indicateur de frappe ('typing' in data) et statut en ligne / hors-ligne diffusés via le channel group.
- Comptage des messages non lus injecté dans tous les templates via context processor (unread_messages_count).
- Marquage automatique des messages comme lus à l'ouverture d'une conversation.

BF5 — Notifications en temps réel

- Notifications push via WebSocket pour 4 types d'événements : like, commentaire, abonnement, message.
- Déclenchement automatique par signaux Django (post_save) sur Like, Comment, Follow et Message.
- Affichage instantané côté client : toast discret (4 secondes) + badge incrémenté dans la barre de navigation.
- Comptage des notifications non lues injecté dans tous les templates via context processor (unread_notifications_count).
- Page de liste complète des notifications avec marquage automatique comme lues à l'ouverture.

BF6 — Stories éphémères

- Publication d'une story (image + légende optionnelle, max 10 Mo) visible 24 heures.
- Expiration automatique : expires_at = timezone.now() + timedelta(hours=24) calculé dans Story.save().
- Filtrage ORM (expires_at__gt=timezone.now()) pour exclure les stories expirées sans tâche planifiée.
- Strip horizontal scrollable dans le fil d'actualité affichant les stories actives des personnes suivies.
- Visionneuse plein écran avec barre de progression JavaScript de 10 secondes et redirection automatique.
- Suppression manuelle par l'auteur avant expiration (soft delete).

BF7 — Recherche

- Recherche d'utilisateurs par nom d'utilisateur ou adresse e-mail (Q(username__icontains=query) | Q(email__icontains=query)).
- Résultats annotés avec is_following pour afficher dynamiquement le bouton Suivre/Abonné.

2.2 Besoins non fonctionnels

Sécurité

- Toutes les vues utilisent SocialLoginRequired (étend LoginRequiredMixin) : un utilisateur non connecté est systématiquement redirigé.
- Tokens CSRF sur tous les formulaires. Configuration CSRF_TRUSTED_ORIGINS pour les requêtes derrière le reverse proxy Coolify/Traefik.
- Échappement automatique des données utilisateur dans les templates Django (protection XSS).
- UUID publics dans toutes les URLs de ressources : jamais d'id séquentiel exposé.

- Mots de passe hachés avec PBKDF2 + SHA-256 via les validateurs Django configurés dans AUTH_PASSWORD_VALIDATORS.

Performance

- Requêtes ORM optimisées avec select_related et prefetch_related pour éviter le problème N+1.
- Annotation SQL avec Exists + OuterRef pour calculer user_liked en une seule requête sans charger tous les likes en mémoire.
- WebSockets pour éviter le polling HTTP périodique côté client, réduisant la charge serveur.
- Fichiers statiques servis avec compression gzip/brotli via WhiteNoise (CompressedStaticFilesStorage).

Maintenabilité

- Application découpée en sept apps Django indépendantes, chacune responsable d'un domaine métier.
- Modèle abstrait BaseModel centralisant les champs d'audit (public_id, created_at, updated_at, created_by, updated_by, is_deleted, deleted_at, deleted_by).
- Utilisation du pattern %(class)s dans les related_name de BaseModel pour éviter les conflits entre classes héritantes.

Utilisabilité

- Interface responsive Bootstrap 5 compatible mobile, tablette et desktop.
- Interactions fréquentes (like, follow) sans rechargement de page (fetch/AJAX + JsonResponse).
- Messages flash Django (messages.success, messages.error) pour le feedback utilisateur.

2.3 Contraintes techniques

- Utilisation obligatoire du framework Django (Python 3.12).
- Utilisation de Django Channels pour la communication temps réel (WebSocket).
- Base de données relationnelle PostgreSQL avec gestion de l'encodage UTF-8.
- Support de l'upload d'images via Pillow (dépendances binaires libjpeg, libpng, zlib dans le Dockerfile).
- Déploiement containerisé sur infrastructure Cloud avec gestion des fichiers statiques.

2.4 Public cible

ConnectX s'adresse à un public jeune et technophile, habitué aux grands réseaux sociaux. Dans sa dimension académique, la plateforme est conçue pour des étudiants et enseignants souhaitant partager du contenu et interagir dans un environnement contrôlé. L'interface, inspirée des standards visuels de Twitter/Instagram, minimise la courbe d'apprentissage grâce à des paradigmes d'interaction familiers : fil d'actualité chronologique, stories en haut de page, notifications en temps réel.

Sur le plan pédagogique, ConnectX constitue également une base de code de référence pour tout développeur Python souhaitant comprendre l'architecture d'un réseau social moderne construit avec Django : la séparation des apps, les patterns de sécurité, l'optimisation ORM et l'intégration WebSocket y sont traités de façon didactique avec des commentaires détaillés dans chaque fichier source.

3. Conception et Architecture

3.1 Architecture globale et modules

ConnectX adopte une architecture modulaire fondée sur la séparation stricte des responsabilités. L'application est découpée en sept apps Django indépendantes, chacune encapsulant un domaine métier précis avec ses propres modèles, vues, URLs, formulaires et (pour chat et notifications) ses consumers WebSocket.

App Django	Responsabilité	Fichiers clés
core	Modèle abstrait d'audit partagé	models.py
accounts	Comptes, profils, auth, email	models.py, views.py, signals.py, forms.py
posts	Publications, images, likes, commentaires	models.py, views.py, forms.py
social	Relations d'abonnement follow/unfollow	models.py, views.py
chat	Messagerie privée (HTTP + WebSocket)	models.py, views.py, consumers.py, routing.py
notifications	Notifications temps réel (WebSocket)	models.py, signals.py, consumers.py
stories	Stories éphémères (24h)	models.py, views.py, forms.py

L'architecture repose sur deux couches de communication complémentaires. La couche HTTP classique (vues CBV, templates Django, ORM) gère les opérations CRUD et la navigation. La couche WebSocket (Django Channels) gère les connexions persistantes pour la messagerie et les notifications. Ces deux couches coexistent grâce à l'entrée ASGI unique définie dans asgi.py via ProtocolTypeRouter, qui dispatche chaque connexion vers la couche appropriée selon son type ('http' ou 'websocket').

La séparation apps/ dans le chemin Python (sys.path.insert(0, str(BASE_DIR / 'apps'))) permet de référencer les apps sans le préfixe 'apps.' dans INSTALLED_APPS et les imports, gardant le code plus lisible. Les templates sont centralisés dans un répertoire templates/ à la racine du projet, avec des sous-dossiers par app.

3.2 Modèle de données détaillé

Le modèle BaseModel — pattern d'audit commun

BaseModel est la pièce maîtresse de l'architecture de données. C'est une classe abstraite Django (Meta: abstract = True) dont héritent tous les modèles de ConnectX, à l'exception du modèle User (qui étend AbstractUser avec ses propres contraintes). Ce pattern est directement inspiré de BaseEntity dans les architectures Java/JPA Spring Data, adapté ici à Django ORM.

Le commentaire dans le code source résume l'intention : « Inspiré du pattern BaseEntity Java (JPA / Spring Data). » BaseModel apporte trois catégories de champs :

- Identité publique : public_id (UUIDField avec db_index=True) exposé dans les URLs à la place de l'id séquentiel interne, jamais exposé.
- Audit temporel : created_at (auto_now_add=True) et updated_at (auto_now=True) pour l'horodatage automatique.
- Audit utilisateur : created_by et updated_by (ForeignKey vers AUTH_USER_MODEL) avec la syntaxe %(class)s dans related_name pour éviter les conflits Django entre toutes les classes héritantes.
- Soft delete : is_deleted (BooleanField), deleted_at (DateTimeField nullable) et deleted_by (ForeignKey) pour la suppression logique sans effacement physique.

Le soft delete mérite une attention particulière : il est utilisé de façon cohérente sur toutes les entités (Post, Like, Follow, Story, Message, Notification). Cela signifie qu'un like supprimé n'est pas effacé de la base mais marqué is_deleted=True. Si l'utilisateur re-like, la vue retrouve l'entrée existante et la réactive (is_deleted=False), évitant ainsi la violation de la contrainte unique_together. Cette approche préserve un historique complet et permet des statistiques d'engagement réelles.

Modèle User et Profile

Le modèle User (accounts/models.py) étend AbstractUser sans hériter de BaseModel (pour des raisons de compatibilité avec l'infrastructure d'authentification Django). Il ajoute : email (unique=True, utilisé comme USERNAME_FIELD), public_id (UUID), et is_online (BooleanField mis à jour par les consumers WebSocket lors des connexions/déconnexions).

Un champ is_email_verified a été ajouté dans la troisième migration (0003_user_is_email_verified.py) lors du développement, illustrant l'évolution incrémentale du schéma. Ce champ booléen est distinct de is_active : is_active contrôle la connexion, is_email_verified trace l'historique de vérification.

Le Profile (OneToOneField vers User, lié avec related_name='profile') stocke : bio, avatar (upload_to='avatars/'), cover_image (upload_to='covers/'), location et website. Il est créé automatiquement par le signal post_save défini dans accounts/signals.py, garantissant qu'aucun utilisateur n'existe sans profil associé, même créé via l'admin ou un script.

Modèles Post, PostImage, Like, Comment

Post stocke content (TextField max 2000) et is_edited (pour les futures modifications). La séparation des images dans PostImage (ForeignKey vers Post avec related_name='images') respecte la première forme normale et permet d'attacher des métadonnées à chaque image individuellement (alt_text pour l'accessibilité).

Like implémente le soft delete particulièrement élaboré : unique_together = ('user', 'post') garantit l'unicité en base, tandis que LikeToggleView gère trois cas distincts : premier like (création), re-like (réactivation d'un like soft-deleted), et unlike (soft delete). Comment est ordonné par created_at ASC pour un affichage chronologique naturel dans les fils de commentaires.

Les propriétés likes_count et comments_count sur Post filtrent sur is_deleted=False pour ne compter que les éléments actifs, évitant les requêtes N+1 dans les templates via les annotations SQL de la FeedView.

Modèle Follow

Follow représente une relation orientée (follower → following) avec unique_together = ('follower', 'following'). Les related_name 'following_relations' (depuis follower) et

'follower_relations' (depuis following) permettent une navigation naturelle depuis un objet User : request.user.following_relations.filter(is_deleted=False) retourne tous les abonnements actifs de l'utilisateur connecté.

Ce modèle alimente directement la requête de feed : following_ids = list(request.user.following_relations.filter(is_deleted=False).values_list('following_id', flat=True)). Puis Post.objects.filter(author_id__in=following_ids + [request.user.id]) permet de récupérer les publications pertinentes en une seule requête SQL.

Modèles Conversation et Message

Conversation utilise ManyToManyField pour les participants, ce qui supporte techniquement les groupes (plus de 2 participants), même si l'interface actuelle ne le propose pas encore. La méthode get_other_participant(user) retourne l'autre participant dans une conversation à deux, utilisée dans les templates pour afficher le nom de l'interlocuteur.

Message est ordonné par created_at ASC pour un affichage de bas en haut (style messagerie). Le champ is_read permet au context processor unread_messages_count (chat/context_processors.py) de comptabiliser les messages non lus et de les injecter dans tous les templates pour le badge de la navbar. À l'ouverture d'une conversation, ConversationDetailView marque automatiquement tous les messages non lus comme lus : messages_qs.filter(is_read=False).exclude(sender=request.user).update(is_read=True).

Modèle Notification

Notification centralise les quatre types d'événements sociaux via une constante de classe (TYPE_LIKE='like', TYPE_COMMENT='comment', TYPE_FOLLOW='follow', TYPE_MESSAGE='message') et des choix CharField (NOTIF_TYPES). Elle référence recipient, sender, notif_type, post (nullable — absent pour follow et message) et is_read.

La méthode get_message() produit le texte localisé en français affiché dans les toasts et la liste des notifications. Le context processor unread_notifications_count (notifications/context_processors.py) fonctionne en parallèle de celui des messages pour maintenir deux badges distincts dans la navbar.

Modèle Story

Story hérite de BaseModel et ajoute expires_at (DateTimeField). La méthode save() calcule expires_at automatiquement à la création (if not self.pk), en ajoutant STORY_DURATION_HOURS = 24 heures à timezone.now(). La propriété is_expired permet une vérification rapide sans requête. La propriété time_remaining retourne le timedelta restant ou None si expirée, utilisable pour des indicateurs de progression.

3.3 Architecture ASGI et WebSocket

La configuration ASGI (connectx/asgi.py) est le cœur de l'architecture temps réel. Elle est construite en trois étapes pour respecter l'ordre d'initialisation de Django :

1. os.environ['PGCLIENTENCODING'] = 'UTF8' est défini avant tout import Django pour corriger l'encodage PostgreSQL sur Windows.
2. django_asgi_app = get_asgi_application() initialise Django (charge settings, modèles, middleware).
3. Seulement après cette initialisation, les consumers Channels peuvent être importés (ils dépendent des modèles Django).

Le routage ASGI est défini par ProtocolTypeRouter qui distingue deux protocoles :

```
application = ProtocolTypeRouter({
    'http': django_asgi_app,
    'websocket': AuthMiddlewareStack(
        URLRouter(chat_ws + notif_ws)
    ),
})
```

AuthMiddlewareStack injecte l'utilisateur Django authentifié dans scope['user'] pour chaque connexion WebSocket, permettant aux consumers de vérifier l'identité sans couche d'authentification supplémentaire. Les routes WebSocket sont définies dans deux fichiers routing.py distincts :

- chat/routing.py : ws/chat/<int:conversation_id>/ → ChatConsumer
- notifications/routing.py : ws/notifications/ → NotificationConsumer

La communication entre signaux Django synchrones et channel layer asynchrone est résolue par async_to_sync d'asgiref. Cette fonction crée temporairement un event loop pour exécuter une coroutine depuis un contexte synchrone, ce qui est exactement le besoin des signaux post_save qui déclenchent les notifications.

3.4 Choix technologiques et justifications

Django 5 + Django Channels

Django 5 est le framework Python web le plus mature, suivant le principe batteries included : ORM, authentification, migrations, formulaires, template engine — tout est intégré et cohérent. Django 5 apporte une meilleure intégration ASGI native et des performances ORM améliorées. Django Channels est la solution officielle pour le WebSocket avec Django, s'intégrant nativement avec l'écosystème sans imposer un framework asynchrone alternatif.

PostgreSQL + Redis

PostgreSQL est choisi pour son support natif des UUID (utilisés comme public_id), ses contraintes d'intégrité avancées (unique_together, NOT NULL) et son excellent backend Django. Redis est utilisé comme channel layer de production (channels_redis). La configuration dans settings.py est conditionnelle : si REDIS_URL est défini (en production), RedisChannelLayer est utilisé ; sinon, InMemoryChannelLayer (développement). Cette dualité permet de développer localement sans Redis tout en garantissant la scalabilité en production.

Daphne + WhiteNoise + Coolify

Daphne (serveur ASGI de référence pour Django Channels) est listé en premier dans INSTALLED_APPS car il doit remplacer le serveur de développement Django avant le chargement de staticfiles. WhiteNoise sert les fichiers statiques directement depuis Django avec compression, évitant la configuration d'un Nginx dédié pour ce rôle. La configuration WHITENOISE_MANIFEST_STRICT = False évite les erreurs sur les fichiers .map manquants. Coolify automatise la gestion des certificats SSL Let's Encrypt, du reverse proxy Traefik et des déploiements continus depuis GitHub.

Jazzmin + Crispy Forms + django-environ

Jazzmin remplace l'interface d'administration Django par défaut avec un thème moderne Bootstrap-AdminLTE, configuré avec une palette rouge cohérente avec l'identité ConnectX (brand_colour: 'navbar-danger', sidebar: 'sidebar-dark-danger'). Crispy Forms rend les

formulaires Bootstrap 5 via CRISPY_TEMPLATE_PACK = 'bootstrap5'. Django-environ gère les variables d'environnement avec typage (env.bool, env.int, env.list, env.db) et lecture automatique du fichier .env.

4. Développement

4.1 Structure du projet et découpage modulaire

Le projet ConnectX est organisé selon la structure suivante :

```
connectx/
├── connectx/          # Configuration principale
│   ├── settings.py    # Paramètres (environ, channels, jazzmin, email…)
│   ├── asgi.py        # Point d'entrée ASGI (HTTP + WebSocket)
│   ├── urls.py        # Routage racine
│   └── wsgi.py        # Point d'entrée WSGI (non utilisé en prod)
├── apps/              # Modules Django
│   ├── core/          # BaseModel abstrait
│   ├── accounts/      # Auth, profils, email
│   ├── posts/         # Publications, images, likes, commentaires
│   ├── social/        # Follow/unfollow
│   ├── chat/          # Messagerie (HTTP + WebSocket)
│   ├── notifications/ # Notifications temps réel
│   └── stories/       # Stories éphémères
├── templates/         # Templates HTML centralisés
├── static/            # Fichiers statiques (CSS, JS, images)
└── media/             # Fichiers uploadés par les utilisateurs
```

Le répertoire apps/ est ajouté au chemin Python via sys.path.insert(0, str(BASE_DIR / 'apps')), permettant de référencer les apps directement par leur nom ('accounts', 'posts'…) sans le préfixe 'apps.'. Chaque app suit la même structure : models.py, views.py, urls.py, admin.py, forms.py, migrations/. Les apps de communication ajoutent consumers.py et routing.py.

4.2 Méthodologie de travail

Le développement a suivi une approche itérative et incrémentale, chaque itération livrant une version testable de l'application :

4. Itération 1 : Mise en place Django 5, modèle User personnalisé (AbstractUser + email comme USERNAME_FIELD), migration initiale, authentification basique.
5. Itération 2 : App core (BaseModel), app posts (Post, PostImage, Like, Comment), templates de feed et de détail.
6. Itération 3 : App social (Follow, FollowToggleView AJAX), app accounts complète (profil, suggestions, recherche).
7. Itération 4 : App chat — messagerie HTTP puis intégration WebSocket avec ChatConsumer, indicateur de frappe.
8. Itération 5 : App notifications — signaux post_save, NotificationConsumer, toasts temps réel. App stories — StoryCreateView, visionneuse plein écran.
9. Itération 6 : Workflow vérification email (is_active=False, token, VerifyEmailView), polissage CSS, admin Jazzmin.
10. Itération 7 : Déploiement Docker sur Hetzner/Coolify, résolution des problèmes de production (Pillow, WhiteNoise, CSRF, SMTP).

Le contrôle de version Git a été utilisé tout au long du projet. La commande python manage.py check a été exécutée systématiquement après chaque modification de modèle pour détecter les conflits de related_name et les erreurs de configuration. La découverte du conflit de related_name dans BaseModel a conduit à l'adoption de la syntaxe %(class)s, solution recommandée dans la documentation Django.

4.3 Module accounts — Authentification et profils

Ce module gère l'ensemble du cycle de vie des comptes utilisateurs. La vue RegisterView illustre bien le pattern du workflow de vérification email :

```
def post(self, request):
    form = RegisterForm(request.POST)
    if form.is_valid():
        user = form.save()
        user.is_active = False  # Désactivé jusqu'à vérification
        user.save()
        uid = urlsafe_base64_encode(force_bytes(user.pk))
        token = default_token_generator.make_token(user)
        # Envoi du lien d'activation par email
        email = EmailMessage(mail_subject, message, to=[user.email])
        email.send(fail_silently=True)
```

La vue CustomLoginView surcharge form_invalid() pour personnaliser les messages d'erreur : elle distingue le cas « compte non activé » (user.is_active=False) du cas « identifiants incorrects ». Cette distinction améliore l'expérience utilisateur tout en restant sécurisée (on ne révèle pas si l'email est enregistré ou non dans le cas générique).

Le signal accounts/signals.py crée automatiquement un Profile à chaque création d'User :

```
@receiver(post_save, sender=User)
def create_user_profile(sender, instance, created, **kwargs):
    if created:
        Profile.objects.create(user=instance)
```

La UserProfileView annote les résultats avec is_following pour que le bouton Suivre/Abonné reflète l'état réel, et utilise select_related('profile') pour éviter les requêtes N+1 sur les avatars.

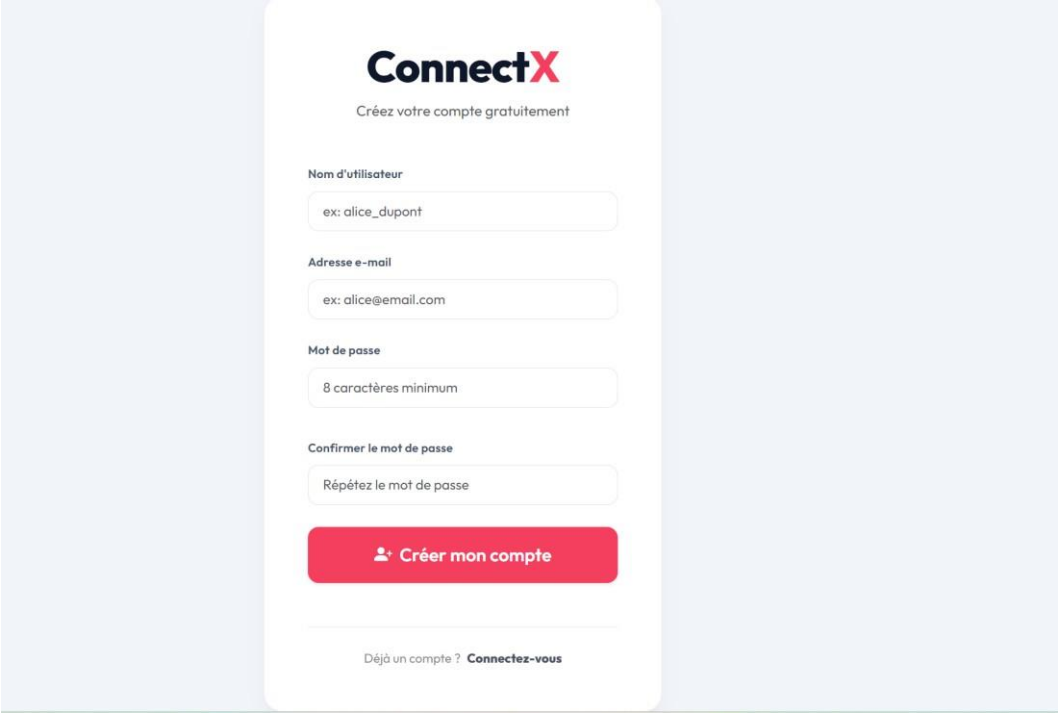

Figure 1 — Formulaire d'inscription : création de compte avec email, nom d'utilisateur et mot de passe

Figure 1 — Formulaire de connexion (gauche) et page d'inscription avec validation (droite)

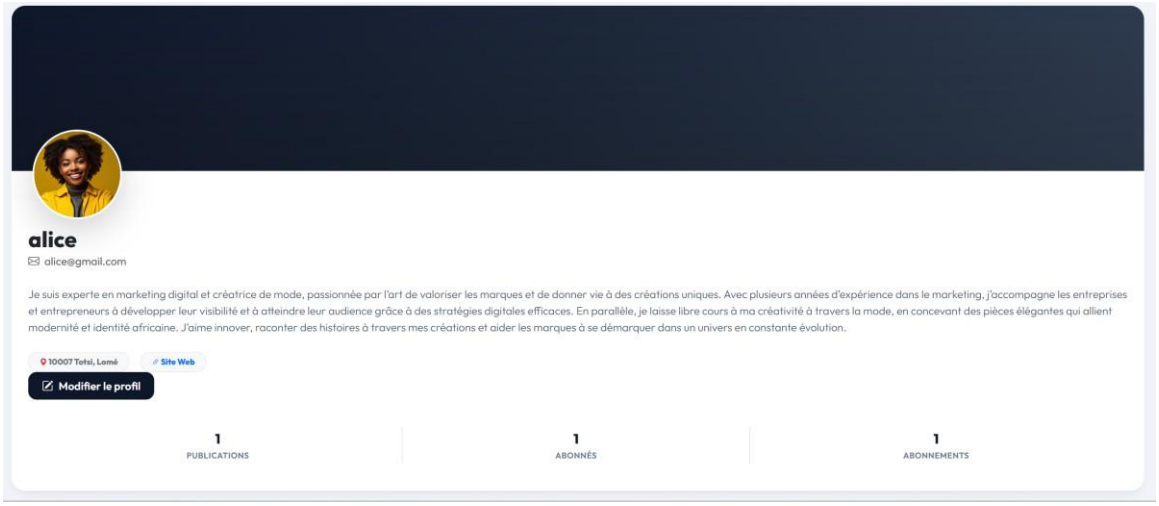

Figure 2 — Page de profil : cover, avatar, bio, statistiques sociales et publications

4.4 Module posts — Publications et fil d'actualité

La FeedView est la vue la plus complexe du projet. Elle illustre plusieurs bonnes pratiques Django ORM avancées :

```
liked_subquery = Like.objects.filter(
    post=OuterRef('pk'), user=request.user, is_deleted=False
)
posts = (
    Post.objects
    .filter(is_deleted=False, author_id__in=following_ids + [request.user.id])
    .select_related('author', 'author__profile')
```

```
    .prefetch_related('images', 'likes', 'comments')
    .annotate(user_liked=Exists(liked_subquery))
    .order_by('-created_at')
)
```

select_related('author', 'author__profile') évite N+1 requêtes sur les avatars en chargeant User et Profile en une seule jointure SQL. prefetch_related charge les images, likes et commentaires en requêtes séparées optimisées plutôt qu'une par post. L'annotation user_liked via Exists+OuterRef calcule en SQL si l'utilisateur a liké chaque post, sans charger tous les likes en mémoire Python.

La LikeToggleView retourne une JsonResponse avec l'état mis à jour et le nouveau compteur, permettant au JavaScript côté client de mettre à jour l'interface sans rechargement :

```
return JsonResponse({
    'liked': liked,
    'count': post.likes.filter(is_deleted=False).count(),
})
```

La validation des images à deux niveaux utilise les constantes MAX_IMAGES = 5, MAX_IMAGE_SIZE = 5 * 1024 * 1024 et ALLOWED_TYPES = {'image/jpeg', 'image/png', 'image/gif', 'image/webp'}, vérifiées sur img.content_type (fourni par le navigateur et vérifié côté serveur).

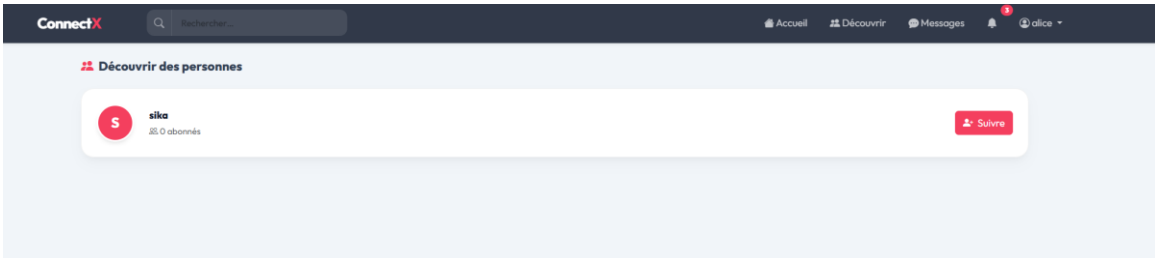

Figure 3 — Fil d'actualité : strip stories en haut, formulaire de publication, posts et sidebar suggestions

Figure 4 — Formulaire de publication avec prévisualisation JavaScript des images sélectionnées

4.5 Module social — Abonnements et découverte

La FollowToggleView gère trois états distincts grâce au soft delete, rendant l'interaction idempotente :

```
follow = Follow.objects.filter(follower=request.user, following=target).first()
if follow:
    if follow.is_deleted:  # Re-follow
        follow.is_deleted = False; follow.deleted_at = None
        follow.deleted_by = None; follow.save(); following = True
    else:  # Unfollow
        follow.is_deleted = True; follow.deleted_at = timezone.now()
        follow.deleted_by = request.user; follow.save(); following = False
else:  # Premier follow
    Follow.objects.create(follower=request.user, following=target, ...);
following = True
```

La réponse JSON inclut le nouveau compteur d'abonnés du profil cible, permettant à l'interface de mettre à jour le chiffre en temps réel. La SuggestionsView exclut les comptes staff et superuser pour ne proposer que de vrais utilisateurs de la plateforme.

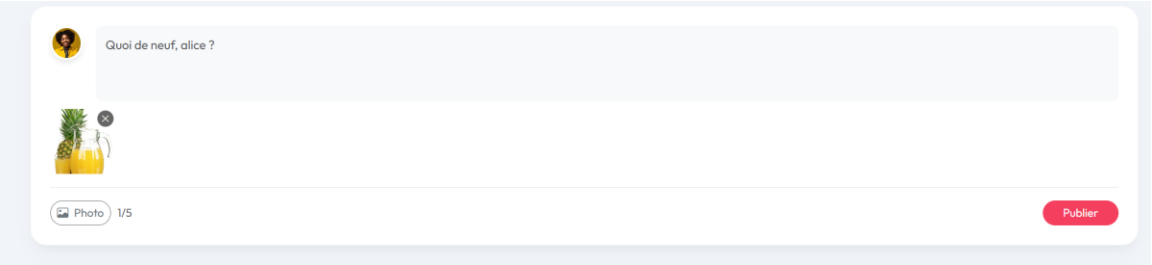

Figure 5 — Page de suggestions : grille des utilisateurs à suivre avec bouton d'abonnement rapide

4.6 Module chat — Messagerie en temps réel

L'architecture de la messagerie combine deux couches. La couche HTTP (ConversationListView, ConversationDetailView, StartConversationView) gère la navigation, l'historique et la création des conversations. StartConversationView implémente le pattern « get_or_create » pour les conversations :

```
conversation = (
    Conversation.objects
    .filter(participants=request.user, is_deleted=False)
    .filter(participants=other_user)
    .first()
)
if not conversation:
    conversation = Conversation.objects.create(created_by=request.user)
    conversation.participants.add(request.user, other_user)
```

La couche WebSocket (ChatConsumer) est un AsyncWebsocketConsumer qui gère plusieurs types de messages dans receive() : les indicateurs de frappe ('typing' in data) sans sauvegarde base de données, et les messages réels avec persistance via database_sync_to_async. Le consumer émet également des événements de statut (user_status) à la connexion et déconnexion pour mettre à jour le indicateur de présence en ligne.

La sécurité est assurée à deux niveaux dans connect() : vérification de l'authentification (scope['user'].is_authenticated) et vérification de la participation à la conversation (is_participant(), requête asynchrone via database_sync_to_async). Un guard protège disconnect() contre le cas où connect() a échoué avant group_add : if not hasattr(self, 'room_group_name'): return.

```
async def connect(self):
    if not self.user.is_authenticated:
```

```
        await self.close(); return
    if not await self.is_participant():
        await self.close(); return
    await self.channel_layer.group_add(self.room_group_name, self.channel_name)
    await self.accept()
```

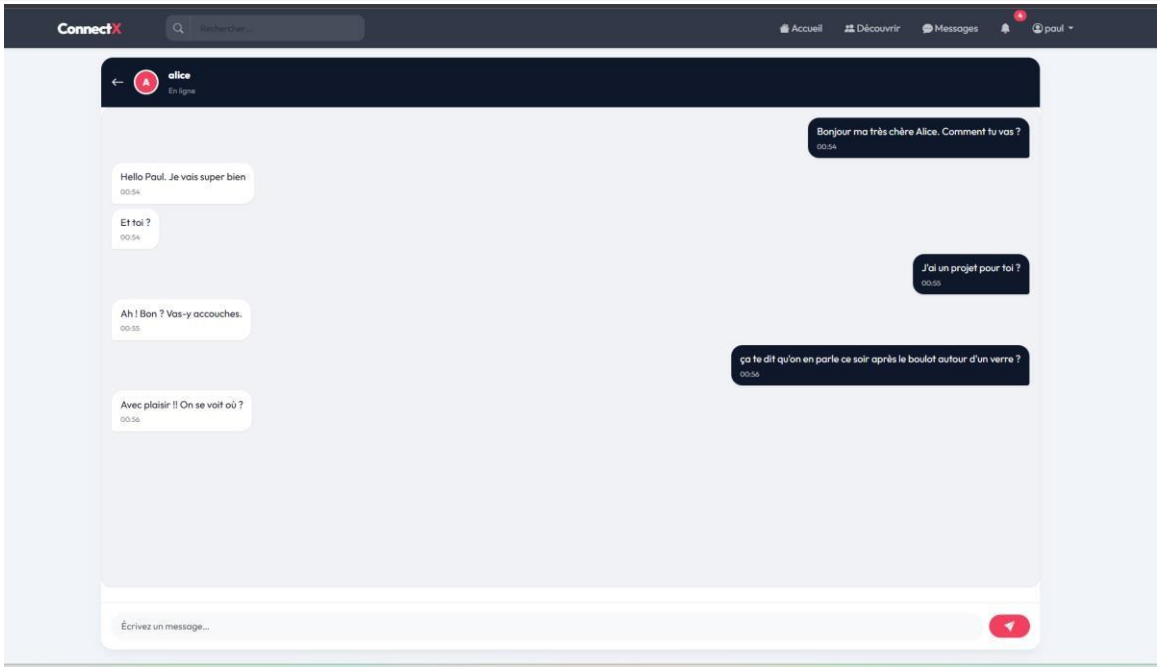

Figure 6 — Interface de messagerie temps réel : bulles de dialogue, indicateur de frappe et statut en ligne

4.7 Module notifications — Alertes instantanées

Le système de notifications repose sur deux composants découplés : les signaux Django (côté serveur) et le NotificationConsumer (côté client WebSocket). Les quatre signaux dans notifications/signals.py sont connectés aux modèles de trois apps différentes (posts.Like, posts.Comment, social.Follow, chat.Message), illustrant la puissance du pattern signal pour le découplage inter-apps.

La fonction push_notification() est le pont entre le monde synchrone des signaux et le monde asynchrone du channel layer :

```
def push_notification(notification):
    channel_layer = get_channel_layer()
    async_to_sync(channel_layer.group_send)(
        f'notif_{notification.recipient_id}',
        {'type': 'new_notification', 'id': notification.id,
         'notif_type': notification.notif_type,
         'message': notification.get_message(),
         'sender': notification.sender.username}
    )
```

Chaque utilisateur connecté rejoignant son groupe personnel notif_{user_id} via NotificationConsumer. Le client JavaScript maintient une connexion WebSocket permanente (dans base.html) et affiche un toast Bootstrap à chaque message reçu, tout en incrémentant le badge de la cloche dans la navbar.

Le signal on_like contient une vérification anti-spam importante : if instance.user == instance.post.author: return — un utilisateur ne reçoit pas de notification lorsqu'il like sa propre publication. La même logique s'applique à on_comment. De plus, les signaux ne se

déclenchent que sur created=True (pas sur les mises à jour de soft delete) pour éviter les notifications parasites lors d'un unlike.

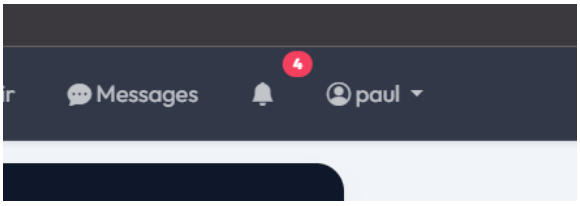

Figure 7 — Toast de notification en temps réel et badge dans la barre de navigation

4.8 Module stories — Contenu éphémère

L'élégance du module stories réside dans sa gestion de l'expiration sans infrastructure externe. La méthode save() calcule expires_at automatiquement :

```
STORY_DURATION_HOURS = 24

def save(self, *args, **kwargs):
    if not self.pk:  # Seulement à la création
        self.expires_at = timezone.now() + timedelta(hours=STORY_DURATION_HOURS)
    super().save(*args, **kwargs)
```

Le filtrage dans FeedView exclut naturellement les stories expirées : Story.objects.filter(expires_at__gt=timezone.now(), ...). Aucune tâche planifiée n'est nécessaire pour masquer les stories expirées — elles disparaissent simplement des requêtes. La propriété time_remaining offre une API pratique pour les barres de progression dans le template.

La validation dans StoryCreateView utilise les constantes MAX_STORY_SIZE = 10 * 1024 * 1024 (10 Mo, plus généreux que les posts car les photos de story sont généralement des images plein format) et les mêmes ALLOWED_TYPES que pour les posts. La visionneuse plein écran inclut une barre de progression JavaScript qui redirige automatiquement vers le feed après 10 secondes.

Figure 8 — Strip stories dans le fil d'actualité (gauche) et visionneuse plein écran avec barre de progression (droite)

4.9 Interface d'administration Jazzmin

L'interface d'administration est configurée avec Jazzmin, un thème AdminLTE3 moderne. La configuration JAZZMIN_SETTINGS et JAZZMIN_UI_TWEAKS dans settings.py définit une identité visuelle cohérente : couleur de marque rouge (brand_colour: 'navbar-danger'), sidebar sombre rouge (sidebar: 'sidebar-dark-danger'), thème Bootstrap Flatly. Les icônes Font Awesome 5 sont assignées à chaque modèle, améliorant la reconnaissance visuelle dans la navigation latérale.

La directive 'daphne' doit impérativement être en premier dans INSTALLED_APPS (avant 'django.contrib.staticfiles') et 'jazzmin' doit précéder 'django.contrib.admin'. Ces contraintes d'ordre sont documentées dans les commentaires du settings.py et constituent un piège classique lors de la configuration.

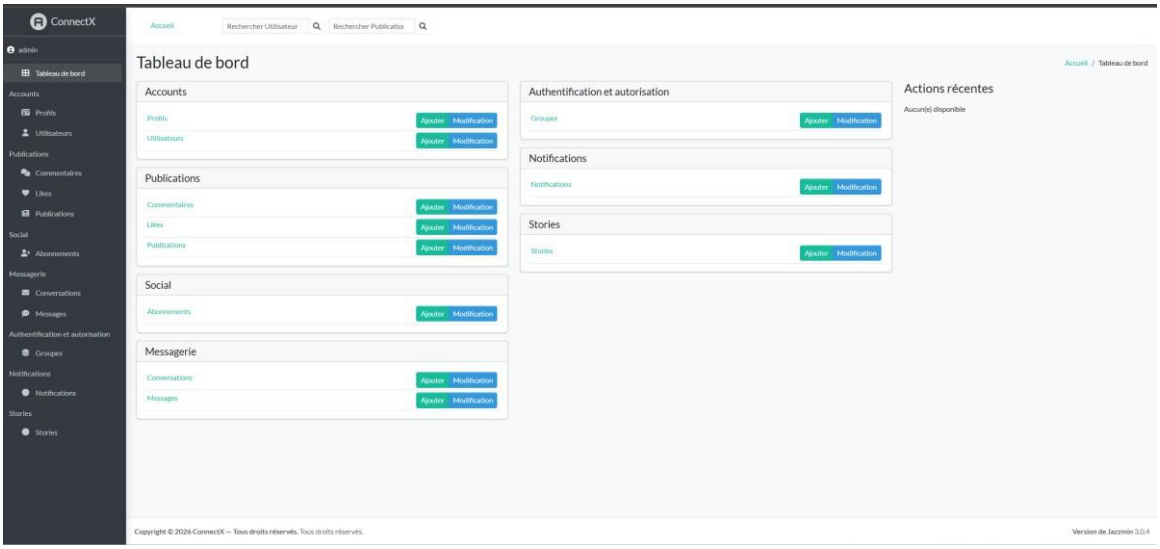

Figure 9 — Interface d'administration Jazzmin avec sidebar, icônes par modèle et thème rouge ConnectX

4.10 Problèmes rencontrés et solutions

Problème 1 — Encodage UTF-8 sur Windows avec PostgreSQL

Symptôme : psycopg2 retournait des UnicodeDecodeError lors du traitement de messages d'erreur PostgreSQL contenant des accents, liés à la locale Windows (CP1252) vs UTF-8 PostgreSQL.

Solution : Définir PGCLIENTENCODING=UTF8 et PGSSLMODE=prefer via os.environ.setdefault() dans settings.py ET dans asgi.py (avant tout import Django), car le serveur ASGI initialise son propre processus.

```
os.environ['PGCLIENTENCODING'] = 'UTF8'
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'connectx.settings')
```

Problème 2 — Signal synchrone ↔ Channel layer asynchrone

Symptôme : Les signaux Django s'exécutent dans un contexte synchrone sans event loop actif. Appeler directement channel_layer.group_send() (une coroutine) depuis un signal provoque RuntimeError: no running event loop.

Solution : async_to_sync(channel_layer.group_send)(...) d'asgiref crée temporairement un event loop pour exécuter la coroutine de façon synchrone. C'est la solution officielle documentée dans Django Channels.

Problème 3 — N+1 queries dans le feed

Symptôme : Une première version sans optimisation ORM générait N+1 requêtes SQL pour N posts (une par avatar d'auteur), soit 50+ requêtes pour un feed de 50 publications.

Solution : select_related('author', 'author__profile') + prefetch_related('images', 'likes', 'comments') + annotation Exists(liked_subquery) réduit à 5-6 requêtes fixes quelle que soit la taille du feed.

Problème 4 — Gestion des fichiers multiples en JavaScript

Symptôme : Sélectionner des images en plusieurs fois écrasait la liste précédente (comportement natif de <input type=file>).

Solution : Utiliser un objet DataTransfer JavaScript comme liste accumulatrice. À chaque nouvelle sélection, les fichiers sont ajoutés à la liste interne (limitée à MAX_IMAGES=5) et un DataTransfer synthétique est reconstruit et réassigné à input.files.

Problème 5 — Conflit de related_name dans BaseModel

Symptôme : Django retournait SystemCheckError : clashing related names sur created_by, updated_by, deleted_by car plusieurs classes héritant de BaseModel définissaient des ForeignKey avec le même related_name.

Solution : Utiliser la syntaxe %(class)s dans related_name (related_name='%(class)s_created', etc.). Django remplace automatiquement %(class)s par le nom de la classe concrète (post_created, follow_created...), garantissant l'unicité.

5. Déploiement et DevOps

ConnectX est accessible en production à l'adresse https://connectx.gfolly.com. Le déploiement couvre l'ensemble de la chaîne : containerisation Docker, infrastructure Cloud Hetzner, orchestration Coolify, gestion du domaine et des certificats SSL.

5.1 Infrastructure Cloud — Hetzner + Coolify

Hetzner Cloud est un fournisseur d'infrastructure cloud européen (Allemagne) offrant un excellent rapport qualité/prix par rapport aux grandes plateformes américaines (AWS, GCP). Le VPS hébergeant ConnectX exécute Coolify, une plateforme d'auto-hébergement open-source comparable à Heroku mais déployée sur votre propre serveur.

Coolify gère automatiquement : le reverse proxy Traefik pour router les requêtes vers les conteneurs, les certificats SSL Let's Encrypt (HTTPS automatique), les déploiements continus depuis GitHub, et les variables d'environnement sécurisées. Cette architecture évite le vendor lock-in des PaaS propriétaires tout en bénéficiant d'une interface de déploiement graphique simple.

La configuration SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https') et USE_X_FORWARDED_HOST = True dans settings.py permettent à Django de reconnaître les requêtes HTTPS transmises par Traefik, garantissant le bon comportement des redirections et des URLs absolues.

5.2 Containerisation Docker

L'application est containerisée avec Docker. Le Dockerfile installe les dépendances binaires nécessaires à Pillow (libjpeg-dev, libpng-dev, zlib1g-dev, libwebp-dev, libpq-dev) avant les packages Python. Sans ces bibliothèques système, l'upload d'images et la connexion PostgreSQL échouent silencieusement en production.

```
FROM python:3.12-slim
RUN apt-get update && apt-get install -y \
    libjpeg-dev libpng-dev zlib1g-dev libwebp-dev libpq-dev gcc \
    && rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
RUN python manage.py collectstatic --noinput
EXPOSE 8000
CMD ["daphne", "-b", "0.0.0.0", "-p", "8000", "connectx.asgi:application"]
```

Le serveur de production est Daphne (pas Gunicorn ni uvicorn), car il est spécialement conçu pour gérer la coexistence HTTP/WebSocket d'un projet Django Channels. Daphne est lancé directement sur l'application ASGI (connectx.asgi:application), qui gère les deux protocoles.

5.3 Configuration des services en production

Channel Layer Redis

En production, la variable d'environnement REDIS_URL active automatiquement RedisChannelLayer dans settings.py :

```
if env('REDIS_URL', default=None):
```

```
    CHANNEL_LAYERS = {'default': {'BACKEND':
'channels_redis.core.RedisChannelLayer',
        'CONFIG': {'hosts': [env('REDIS_URL')]}}}
else:
    CHANNEL_LAYERS = {'default': {'BACKEND':
'channels.layers.InMemoryChannelLayer'}}
```

Redis est essentiel en production multi-processus : InMemoryChannelLayer ne partage pas l'état entre les processus Daphne, ce qui empêcherait les messages WebSocket de fonctionner avec plusieurs workers.

Configuration SMTP Brevo

Brevo (ex-Sendinblue) est le service SMTP utilisé pour les emails d'activation et de réinitialisation de mot de passe. La configuration sépare EMAIL_HOST_USER (identifiant numérique technique Brevo, utilisé pour l'authentification SMTP) de DEFAULT_FROM_EMAIL (adresse affichée dans les emails reçus : 'ConnectX <noreply@gfolly.com>'). Sans cette séparation, les emails peuvent être rejetés ou marqués comme spam.

WhiteNoise et fichiers statiques

WhiteNoise (middleware en second dans MIDDLEWARE, après SecurityMiddleware) intercepte les requêtes de fichiers statiques avant Django. CompressedStaticFilesStorage génère des variantes .gz et .br lors de collectstatic, servant automatiquement la version compressée aux navigateurs compatibles. WHITENOISE_MANIFEST_STRICT = False est nécessaire pour ignorer les fichiers .map référencés dans les CSS/JS minifiés des bibliothèques tierces.

5.4 Problèmes de déploiement résolus

- Pillow sans dépendances binaires : Dockerfile sans libjpeg-dev provoquait des ImportError silencieux sur Pillow. Résolu par l'ajout des dépendances système dans l'étape RUN apt-get.
- CSRF derrière Traefik : Les requêtes POST retournaient 403 Forbidden car Django ne reconnaissait pas le domaine de production. Résolu par CSRF_TRUSTED_ORIGINS = ['https://connectx.gfolly.com'] et la configuration du proxy SSL header.
- Fichiers .map manquants : WhiteNoise plantait lors de collectstatic sur des fichiers .map référencés mais absents. Résolu par WHITENOISE_MANIFEST_STRICT = False.
- Ordre INSTALLED_APPS : Daphne doit être avant staticfiles, Jazzmin avant admin. Une mauvaise ordre causait des erreurs d'import au démarrage.

6. Sécurité de l'Application

La sécurité de ConnectX s'articule autour de quatre axes principaux : l'authentification robuste, la protection des formulaires, la sécurité des connexions WebSocket, et la prévention de l'énumération des ressources.

6.1 Workflow de vérification d'email

Le workflow implémenté dans RegisterView suit le protocole standard de double opt-in :

11. Le compte est créé avec is_active = False, rendant toute connexion impossible.
12. Un token est généré par default_token_generator.make_token(user) (Django PasswordResetTokenGenerator), combinant le pk de l'utilisateur, son mot de passe hashé et un timestamp. Ce token est invalide dès que le mot de passe change.
13. Le lien d'activation encode l'identifiant utilisateur en base64 (urlsafe_base64_encode(force_bytes(user.pk))) et contient le token.
14. VerifyEmailView décode l'identifiant (urlsafe_base64_decode), récupère l'utilisateur, vérifie le token (check_token), puis passe is_active = True et is_email_verified = True.

La robustesse du workflow est assurée par le try/except dans VerifyEmailView qui gère tous les cas d'erreur possibles (TypeError, ValueError, OverflowError, User.DoesNotExist) en une seule clause, évitant les fuites d'information sur la structure interne.

La vue CustomLoginView améliore l'expérience en distinguant les erreurs : si l'email existe mais is_active=False, un message spécifique invite à vérifier sa boîte mail. Sinon, un message générique « Email ou mot de passe incorrect » est affiché, sans révéler si l'email est enregistré (résistance à l'énumération d'emails).

6.2 Gestion et protection des mots de passe

La validation des mots de passe utilise les quatre validateurs Django configurés dans AUTH_PASSWORD_VALIDATORS : similarité avec les attributs utilisateur, longueur minimale, liste des mots de passe communs (fichier de 20 000 entrées), et rejet des mots de passe entièrement numériques. Ces validateurs sont actifs à l'inscription et au changement de mot de passe.

La réinitialisation utilise les vues génériques Django (PasswordResetView, PasswordResetConfirmView) avec des templates personnalisés. Les tokens de réinitialisation sont à usage unique et liés au hash du mot de passe actuel : si un attaquant intercepte un lien après son utilisation, il est déjà invalidé. La configuration dans accounts/urls.py délègue entièrement cette responsabilité aux vues Django testées et auditées.

6.3 Protections CSRF, XSS et reverse proxy

La protection CSRF est assurée par CsrfViewMiddleware (middleware Django par défaut). La configuration spécifique pour le déploiement derrière Coolify/Traefik ajoute CSRF_TRUSTED_ORIGINS dynamiquement à partir de ALLOWED_HOSTS, plus les domaines de production explicites. Sans cette configuration, toutes les requêtes POST retournent 403 Forbidden en production.

La protection XSS est garantie par l'autoescaping de Django dans les templates. Toute variable affichée via {{ variable }} est automatiquement encodée en HTML entities. Les appels

AJAX retournent du JSON, jamais du HTML non échappé. Les données affichées via JavaScript utilisent textContent (non innerHTML) pour éviter l'injection DOM.

Le header SECURE_PROXY_SSL_HEADER = ('HTTP_X_FORWARDED_PROTO', 'https') est essentiel pour que Django génère des URLs absolues en HTTPS (dans les emails d'activation notamment) lorsqu'il s'exécute derrière un reverse proxy qui termine le SSL.

6.4 Sécurité WebSocket et UUID publics

Chaque consumer WebSocket implémente une double vérification en connect() : authentification (scope['user'].is_authenticated) et autorisation spécifique (participation à la conversation pour ChatConsumer). En cas d'échec, self.close() est appelé immédiatement sans accept() préalable. Le guard dans disconnect() (if not hasattr(self, 'room_group_name'): return) protège contre les erreurs dans les cas où connect() a échoué avant d'atteindre group_add.

L'utilisation systématique des UUID publics (public_id) dans toutes les URLs de ressources (publications, conversations, stories) prévient l'énumération séquentielle. Un attaquant ne peut pas explorer les ressources en incrémentant un compteur : chaque UUID est aléatoire et imprévisible (128 bits d'entropie). L'identifiant séquentiel interne (id) n'est jamais exposé dans les réponses HTTP ou les templates.

Le mixin SocialLoginRequired est utilisé sur toutes les vues sociales. Il étend LoginRequiredMixin (qui redirige vers LOGIN_URL = 'accounts:login') et ajoute la redirection automatique des comptes staff vers /admin/ lors de la connexion, séparant clairement les interfaces utilisateur et administrateur.

7. Tests et Validation

7.1 Méthodologie de test

Les tests de ConnectX ont été réalisés selon trois approches complémentaires : vérification statique par Django, tests fonctionnels manuels avec scénarios utilisateur complets, et tests de sécurité ciblés.

Vérification système Django (manage.py check)

La commande python manage.py check a été exécutée après chaque modification de modèle ou de configuration. Elle analyse statiquement les modèles, les migrations, les related_name, les validateurs et les paramètres de sécurité. Elle a permis de détecter plusieurs problèmes tôt : conflits de related_name dans BaseModel (résolu par %(class)s), avertissement W004 sur STATICFILES_DIRS inexistant (résolu par os.makedirs), et erreurs de configuration ASGI.

Tests fonctionnels manuels

Chaque fonctionnalité a été testée avec plusieurs comptes utilisateurs de test, couvrant le chemin nominal (happy path) et les cas d'erreur principaux. Les tests WebSocket ont utilisé deux onglets de navigateur simultanément pour vérifier la diffusion en temps réel.

Tests de sécurité

Accès non authentifié aux pages sociales (→ redirection /login/), tentative de suppression d'un post appartenant à un autre utilisateur (→ 404 via get_object_or_404(Post, author=request.user)), connexion WebSocket sans authentification (→ close() immédiat), connexion WebSocket à une conversation sans participation (→ close() immédiat).

7.2 Résultats des tests fonctionnels

Fonctionnalité	Scénario testé	Résultat
Inscription	Email valide + mdp fort	✓ Email d'activation envoyé
Inscription	Email déjà utilisé	✓ Erreur formulaire affichée
Inscription	Connexion avant activation	✓ Message explicatif (is_active=False)
Connexion	Email + mdp corrects	✓ Redirection feed
Connexion	Mdp incorrect	✓ Message générique sécurisé
Vérification email	Clic sur lien valide	✓ is_active=True, connexion possible
Réinit. mdp	Lien cliqué 2 fois	✓ 2e clic refusé (token invalide)
Publication	Texte + 3 images JPEG	✓ Créée, affiché dans le feed
Publication	7 images (> max)	✓ Erreur côté serveur
Publication	Image > 5 Mo	✓ Erreur serveur retournée
Like / Unlike	Double like/unlike rapide	✓ Compteur AJAX cohérent
Soft delete Like	Unlike puis re-like	✓ Historique préservé

Fonctionnalité	Scénario testé	Résultat
Follow/Unfollow	Toggle multiple fois	✓ Soft delete cohérent
Auto-follow	Se suivre soi-même	✓ Bloqué (status 400)
Chat WebSocket	Envoi/réception temps réel	✓ Réussi (2 onglets)
Chat WebSocket	Indicateur de frappe	✓ Affiché chez le destinataire
Chat WebSocket	Connexion sans auth	✓ Refusée (close())
Notifications	Like → toast destinataire	✓ Reçu en temps réel
Notifications	Pas de notif auto-like	✓ Signal ignoré (même user)
Stories	Expiration 24h	✓ Disparaît du feed
Stories	Accès story expirée	✓ Redirection + message d'avertissement
Sécurité	Suppression post d'autrui	✓ 404 (author dans get_object_or_404)
Admin	Connexion staff → admin	✓ Redirection /admin/ automatique

7.3 Améliorations possibles

Le projet dispose d'une base solide mais plusieurs améliorations pourraient renforcer la qualité du code et les fonctionnalités de la plateforme :

Tests automatisés

L'absence de tests unitaires et d'intégration automatisés est la principale lacune technique. Il serait pertinent d'ajouter des TestCase Django pour chaque vue (authentification, CRUD, permissions) et chaque modèle (contraintes, soft delete). Les consumers WebSocket nécessitent ChannelsLiveServerTestCase fourni par Django Channels pour simuler les connexions WebSocket dans un contexte de test.

Pagination du feed

Le fil d'actualité charge toutes les publications en une seule requête. Sur une base volumineuse, cela deviendrait problématique. La pagination Django (Paginator) ou la pagination infinie (scroll infini via AJAX) sont des améliorations prioritaires pour la scalabilité.

Nettoyage des stories expirées

Les stories expirées restent en base de données (elles sont simplement exclues des requêtes). Un nettoyage périodique via Celery Beat ou django-crontab (DELETE WHERE expires_at < NOW()) libérerait l'espace disque et réduirait la taille des tables.

Conversations de groupe et partage

Le modèle Conversation supporte techniquement les groupes via ManyToMany. L'interface ne l'expose pas encore. De même, les fonctionnalités de partage de publications, de mentions (@username) et de hashtags sont des extensions naturelles de l'architecture actuelle, sans modification du schéma de base pour la plupart.

API REST

Une API REST (Django REST Framework) permettrait de connecter une application mobile (React Native, Flutter) en utilisant les mêmes modèles et permissions. L'architecture actuelle est bien préparée pour cette évolution : les vues AJAX (JsonResponse) préfigurent les endpoints API, et les UUID publics constituent des identifiants stables pour une API.

8. Conclusion et Perspectives

8.1 Bilan global du projet

ConnectX est un projet ambitieux qui a atteint l'ensemble de ses objectifs fonctionnels et techniques. La plateforme déployée à https://connectx.gfolly.com implémente un réseau social complet avec sept modules fonctionnels, couvrant l'authentification sécurisée, les publications multimédia, les abonnements asymétriques, la messagerie temps réel, les notifications push, et les stories éphémères.

Sur le plan technique, le projet a été l'occasion de maîtriser des concepts avancés rarement abordés dans les projets académiques de ce niveau : l'architecture ASGI et la coexistence HTTP/WebSocket, le pattern async/await dans les consumers Django Channels, le pont async_to_sync entre les signaux synchrones et le channel layer asynchrone, et l'optimisation des requêtes ORM avec les sous-requêtes corrélées (Exists + OuterRef).

Le pattern BaseModel inspiré de Spring Data/JPA démontre l'interopérabilité des bonnes pratiques entre les ecosystèmes Java et Python. Le soft delete, les UUID publics, les champs d'audit et la séparation entre identifiant interne et identifiant public sont des pratiques professionnelles directement transposables en environnement industriel.

Le déploiement sur Hetzner Cloud via Coolify a été une expérience DevOps complète, exposant aux problématiques réelles de la mise en production : gestion des dépendances binaires dans Docker, configuration du reverse proxy pour CSRF, séparation des credentials SMTP, et configuration du channel layer Redis. Ces problèmes, absents du développement local, sont typiques des projets en production réelle.

8.2 Pistes d'amélioration et évolutions possibles

ConnectX constitue une base solide pour plusieurs évolutions :

- Infrastructure : Redis Channel Layer avec scaling horizontal, CDN pour les fichiers media (AWS S3 ou Cloudflare R2), pagination du feed.
- Fonctionnalités : partage de publications, mentions (@username), hashtags avec pages dédiées, réactions variées (emojis), conversations de groupe.
- Modération : signalement de contenu, système de blocage d'utilisateurs, tableau de bord de modération dans l'admin.
- API REST (DRF) pour application mobile React Native ou Flutter, avec authentification JWT.
- Tests automatisés : couverture complète avec TestCase Django (vues, modèles, permissions) et ChannelsLiveServerTestCase pour les consumers WebSocket.
- Accessibilité : conformité WCAG 2.1, navigation clavier complète, textes alternatifs sur les images uploadées.

Ce projet a démontré que Django, loin d'être limité aux applications CRUD simples, est capable de supporter une architecture temps réel sophistiquée grâce à Django Channels. La rigueur dans la conception du modèle de données (BaseModel, soft delete, UUID publics) et dans la gestion de la sécurité (vérification email, WebSocket, CSRF) place ConnectX au niveau des standards professionnels du développement web moderne.

Annexes

Annexe A — core/models.py — BaseModel

```
import uuid
from django.conf import settings
from django.db import models

class BaseModel(models.Model):
    public_id  = models.UUIDField(default=uuid.uuid4, editable=False,
unique=True, db_index=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    created_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True,
                   on_delete=models.SET_NULL, related_name='%(class)s_created',
editable=False)
    updated_by = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True,
                   on_delete=models.SET_NULL, related_name='%(class)s_updated',
editable=False)
    is_deleted  = models.BooleanField(default=False)
    deleted_at  = models.DateTimeField(null=True, blank=True)
    deleted_by  = models.ForeignKey(settings.AUTH_USER_MODEL, null=True,
blank=True,
                   on_delete=models.SET_NULL, related_name='%(class)s_deleted',
editable=False)
    class Meta:
        abstract = True
```

Annexe B — connectx/asgi.py — Configuration ASGI

```
import os
os.environ['PGCLIENTENCODING'] = 'UTF8'
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'connectx.settings')
from django.core.asgi import get_asgi_application
django_asgi_app = get_asgi_application()  # Initialise Django AVANT les imports
Channels
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
from chat.routing import websocket_urlpatterns as chat_ws
from notifications.routing import websocket_urlpatterns as notif_ws
application = ProtocolTypeRouter({
    'http': django_asgi_app,
    'websocket': AuthMiddlewareStack(URLRouter(chat_ws + notif_ws)),
})
```

Annexe C — chat/consumers.py — ChatConsumer (extrait)

```
class ChatConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.conversation_id =
self.scope['url_route']['kwargs']['conversation_id']
        self.room_group_name = f'chat_{self.conversation_id}'
        self.user = self.scope['user']
        if not self.user.is_authenticated: await self.close(); return
        if not await self.is_participant(): await self.close(); return
        await self.channel_layer.group_add(self.room_group_name,
self.channel_name)
        await self.accept()
        await self.channel_layer.group_send(self.room_group_name,
```

```
            {'type': 'user_status', 'username': self.user.username, 'online':
True})

    async def receive(self, text_data):
        data = json.loads(text_data)
        if 'typing' in data:  # Indicateur de frappe (pas de sauvegarde BDD)
            await self.channel_layer.group_send(self.room_group_name,
                {'type': 'typing_event', 'username': self.user.username,
'typing': data['typing']})
            return
        content = data.get('message', '').strip()
        if not content: return
        message = await self.save_message(content)
        await self.channel_layer.group_send(self.room_group_name,
            {'type': 'chat_message', 'message': content,
             'sender': self.user.username, 'sender_id': self.user.id,
             'message_id': message.id, 'created_at':
message.created_at.isoformat()})

    @database_sync_to_async
    def is_participant(self):
        return Conversation.objects.filter(
            id=self.conversation_id, participants=self.user,
is_deleted=False).exists()
```

Annexe D — notifications/signals.py — Signaux de notifications

```
from asgiref.sync import async_to_sync
from channels.layers import get_channel_layer
from django.db.models.signals import post_save
from django.dispatch import receiver

def push_notification(notification):
    channel_layer = get_channel_layer()
    async_to_sync(channel_layer.group_send)(
        f'notif_{notification.recipient_id}',
        {'type': 'new_notification', 'id': notification.id,
         'notif_type': notification.notif_type,
         'message': notification.get_message(),
         'sender': notification.sender.username})

@receiver(post_save, sender='posts.Like')
def on_like(sender, instance, created, **kwargs):
    if not created: return           # Ignorer les mises à jour (soft delete)
    if instance.user == instance.post.author: return  # Pas de notif auto-like
    notif = Notification.objects.create(recipient=instance.post.author,
        sender=instance.user, notif_type=Notification.TYPE_LIKE,
        post=instance.post, created_by=instance.user)
    push_notification(notif)
```

Annexe E — stories/models.py — Expiration automatique

```
STORY_DURATION_HOURS = 24

class Story(BaseModel):
    author     = models.ForeignKey(User, on_delete=CASCADE,
related_name='stories')
    image      = models.ImageField(upload_to='stories/')
    caption    = models.CharField(max_length=200, blank=True)
    expires_at = models.DateTimeField()
```

```
    def save(self, *args, **kwargs):
        if not self.pk:
            self.expires_at = timezone.now() +
timedelta(hours=STORY_DURATION_HOURS)
        super().save(*args, **kwargs)

    @property
    def is_expired(self):
        return timezone.now() > self.expires_at

    @property
    def time_remaining(self):
        delta = self.expires_at - timezone.now()
        return delta if delta.total_seconds() > 0 else None
```

Annexe F — posts/views.py — Optimisation FeedView

```
liked_subquery = Like.objects.filter(
    post=OuterRef('pk'), user=request.user, is_deleted=False
)
posts = (
    Post.objects
    .filter(is_deleted=False, author_id__in=following_ids + [request.user.id])
    .select_related('author', 'author__profile')        # Évite N+1 sur avatars
    .prefetch_related('images', 'likes', 'comments')    # Charge en 3 requêtes
    .annotate(user_liked=Exists(liked_subquery))        # Booléen SQL, pas
Python
    .order_by('-created_at')
)
```

Annexe G — connectx/settings.py — Extraits clés

Configuration du channel layer conditionnel (dev/prod) :

```
if env('REDIS_URL', default=None):
    CHANNEL_LAYERS = {'default': {'BACKEND':
'channels_redis.core.RedisChannelLayer',
                                  'CONFIG': {'hosts': [env('REDIS_URL')]}}}
else:
    CHANNEL_LAYERS = {'default': {'BACKEND':
'channels.layers.InMemoryChannelLayer'}}
```

Context processors injectés dans tous les templates :

```
'context_processors': [
    'django.template.context_processors.request',
    'django.contrib.auth.context_processors.auth',
    'django.contrib.messages.context_processors.messages',
    'chat.context_processors.unread_messages_count',
    'notifications.context_processors.unread_notifications_count',
]
```

Gestion des fichiers statiques avec WhiteNoise :

```
STORAGES = {
    'default': {'BACKEND': 'django.core.files.storage.FileSystemStorage'},
    'staticfiles': {'BACKEND':
'whitenoise.storage.CompressedStaticFilesStorage'},
}
WHITENOISE_MANIFEST_STRICT = False  # Ignore les .map manquants
```